



Object Oriented Software Development

Interfaces



Agenda & Reading

► Topics:

- Interfaces
- Abstract Classes VS Interfaces
- Extending Interfaces
- Final variables, methods, classes

► Reading

- Java how to program Late objects version (D & D)
 - Chapter 10
- The Java Tutorial:
 - [Abstract Methods and Classes](http://docs.oracle.com/javase/tutorial/java/landl/abstract.html)
 - <http://docs.oracle.com/javase/tutorial/java/landl/abstract.html>
 - Writing [Final](http://docs.oracle.com/javase/tutorial/java/landl/final.html) Classes and Methods
 - <http://docs.oracle.com/javase/tutorial/java/landl/final.html>
 - [Interfaces](http://docs.oracle.com/javase/tutorial/java/landl/createinterface.html)
 - <http://docs.oracle.com/javase/tutorial/java/landl/createinterface.html>



3. Java Interfaces

Introduction

- ▶ If an abstract class contains only abstract method declarations, it should be implemented as an interface instead.
- ▶ Interfaces offer a capability requiring that unrelated classes implement a set of common methods.
- ▶ Java Interface
 - ▶ Defines a protocol of behaviour that can be implemented
 - ▶ Defines a set of methods but does not implement them (no actual implementation)
 - ▶ is never directly instantiated
- ▶ The interface specifies **what** operations a radio must permit users to perform but does not specify **how** the operations are performed.
- ▶ A Java interface describes a set of methods that can be called on an object.



3. Java Interfaces

Create an Interface

- ▶ An interface declaration begins with the keyword **interface** and contains only constants and abstract methods.
 - ▶ All interface members must be public.
 - ▶ Interfaces may not specify any implementation details, such as concrete method declarations and instance variables.
 - ▶ All methods declared in an interface are implicitly public abstract methods.
 - ▶ All fields are implicitly public, static and final.
 - ▶ Example:
 - ▶ Interface in everyday life: VCR



```
public interface VCR {  
    public boolean loadTape(String tapeName);  
    public boolean eject();  
    public boolean play();  
    public boolean pause();  
    public boolean stop();  
    ...  
    public static final int PLAYING=0, PAUSED=1, STOPPED=2,  
        FORWARD=3, BACKWARD=4, NOTAPE=5;  
}
```



3. Java Interfaces

Use an interface

```
public interface Bounceable {  
    double GRAVITY = 9.8;  
    void hitWall(String wallName);  
}
```

Methods are implicitly abstract

- ▶ To use an interface, a concrete class must specify that it **implements** the interface and must declare each method in the interface with specified signature.
 - ▶ Add the **implements** keyword and the name of the interface to the end of your class declaration's first line.
- ▶ A class that does not implement all the methods of the interface is an abstract class and must be declared **abstract**.
- ▶ Implementing an interface is like signing a contract with the compiler that states, "I will declare all the methods specified by the interface or I will declare my class abstract."

```
public class Basketball implements Bounceable {  
    public void hitWall(String wallName) {  
        System.out.println("I hit " + wallName);  
    }  
    public static void main(String[] args) {  
        Basketball b = new Basketball();  
        b.hitWall("Wall A");  
    }  
}
```



3. Java Interfaces is-a relationship

- ▶ Once a class **implements** an interface, must provide an implementation of **every method** defined within the interface.
 - ▶ A class may implement some additional methods (but these extra methods aren't accessible through this interface)
- ▶ All objects of that class have an **is-a** relationship with the interface type, and all objects of the class are guaranteed to provide the functionality described by the interface.
 - ▶ This is true of all subclasses of that class as well. *extends A*
- ▶ A class can implement many interfaces. *implements I, J, K, L*
- ▶ In Java, classes are less extendible than interfaces, because a Class can extend at most one other Class ("single inheritance"). *Java*
- ▶ Beware: adding another method to an existing Interface will "break" every current implementation of this Interface!
 - ▶ Extension is the preferred way to add new methods to an Interface



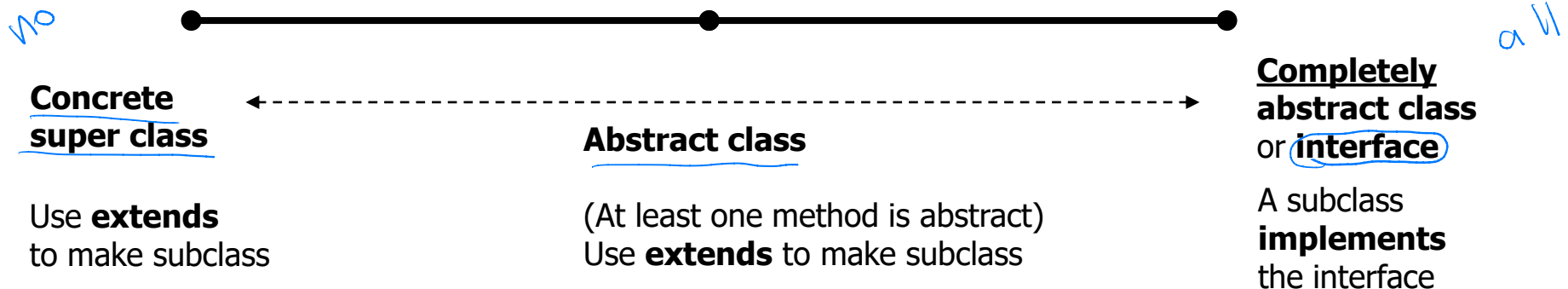
3. Java Interfaces

Abstract classes Vs Interfaces

► Abstract classes Vs Interfaces

- An interface cannot implement any methods, whereas an abstract class can. (i.e. An interface is where all the methods are abstract)
- A class that implements an interface must either provide definitions for all methods or declare itself abstract
- Classes can extend only one class, but can implement one or more interfaces
- Unrelated classes can implement the same interface

► Interface Continuum





4. Interface in unrelated classes

- ▶ An interface is often used when disparate classes (i.e., unrelated classes) need to share common methods and constants.
 - ▶ Allows objects of unrelated classes to be processed polymorphically by responding to the same method calls.
 - ▶ You can create an interface that describes the desired functionality, then implement this interface in any classes that require that functionality.

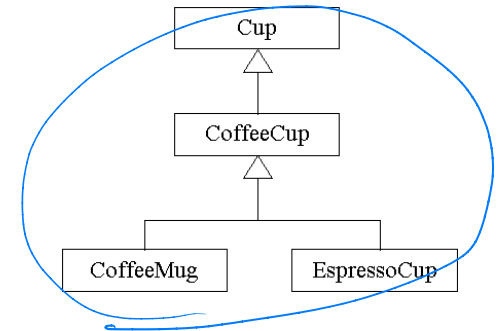


4. Interface in unrelated classes

Design 1

- ▶ Given a family of classes, polymorphism allows you to treat a subclass object as if it were a superclass object
 - ▶ create a single method that could wash any kind of cup in your virtual café.

```
public static void prepareACup(Cup cup) {  
    ...  
    cup.wash() ;  
    ...  
}
```



- ▶ The prepareACup() method can invoke wash() on many different objects

```
class Cup {  
    public void wash() {  
        ...  
    }  
}
```

```
class EspressoCup extends CoffeeCup {  
    public void wash() {  
        ...  
    }  
}
```

```
class CoffeeCup extends Cup {  
    public void wash() {  
        ...  
    }  
}
```

```
class CoffeeMug extends CoffeeCup {  
    public void wash() {  
        ...  
    }  
}
```



4. Interface in unrelated classes

Design 2: Washable?

- ▶ What if you wanted to have a method that can wash any kind of object ("washable" object)?
 - ▶ For example: wash a window, wash your car, or wash a dog
 - ▶ Objects don't seem to fit into the same family
 - ▶ You may want to...

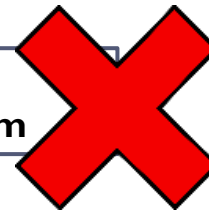
```
class Window {  
    public void wash() {  
        ...  
    }  
}
```

```
class Dog {  
    public void wash() {  
        ...  
    }  
}
```

```
class Car {  
    public void wash() {  
        ...  
    }  
}
```

```
class Cleaner {  
    public static void cleanAnObject(Object obj) {  
        if (obj instanceof Cup) {  
            ((Cup) obj).wash();  
        } else if (obj instanceof Dog) {  
            ((Dog) obj).wash();  
        } else if ...  
    }  
}
```

doesn't use
polymorphism





4. Interface in unrelated classes

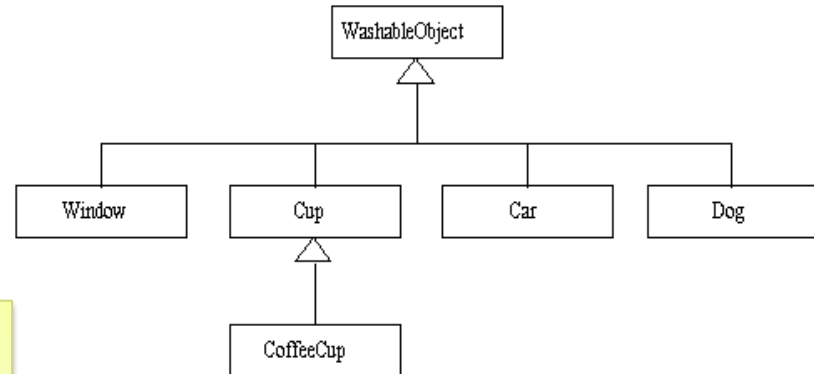
Design 3: WashableObject family

- ▶ Four classes -- cups, cars, windows, and dogs--are combined into the WashableObject family
 - ▶ Full benefit of polymorphism in the cleanAnObject() method

```
abstract class WashableObject {  
    public abstract void wash();  
}
```

```
class Window extends WashableObject {  
    public void wash() {  
        ...  
    }  
}
```

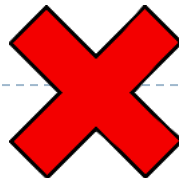
```
class Dog extends WashableObject {  
    public void wash() {  
        ...  
    }  
}
```



```
class Cleaner {  
    public static void cleanAnObject(WashableObject wo) {  
        //...  
        wo.wash();  
    }  
}
```

- ▶ However ..

- ▶ What exactly is a washable object? What does one look like?
Washable is an adjective, not a noun. It describes a behaviour exhibited by objects, not an object itself!

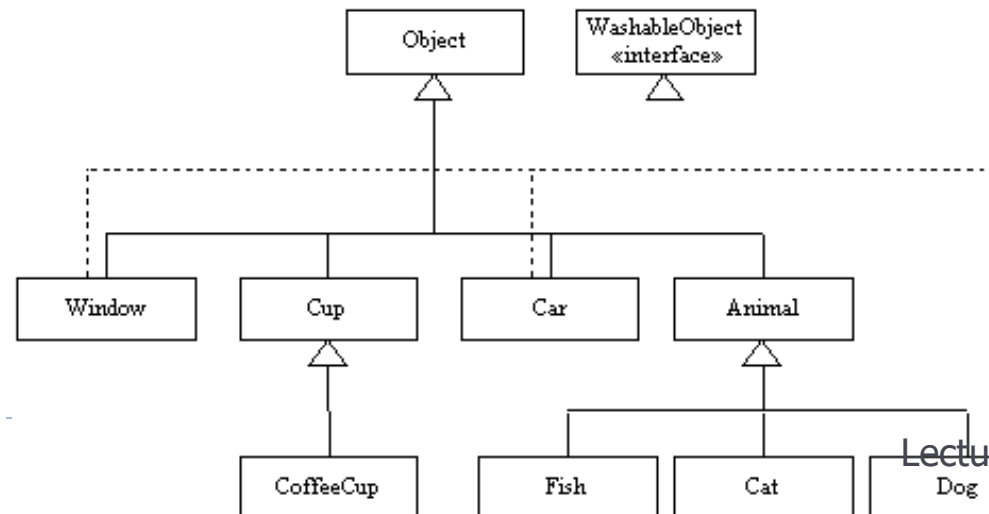




4. Interface in unrelated classes

Design 4: Washable Interface

- ▶ Interfaces in Java allow you to get the benefits of polymorphism without requiring you to build a singly-inherited family of classes.
- ▶ Although a class can extend only one other class, it can "implement" multiple interfaces.
- ▶ Solution
 - ▶ Family of classes for cups, another for animals, one for vehicles (including cars), one for parts of a building etc
 - ▶ Then each washable class can implement the Washable interface.





Exercise 1

- ▶ Consider the following definitions:

```
interface I {  
    public void iMethod1(int i);  
}  
abstract class J implements I {  
    public abstract void jMethod1(int j);  
}
```

- ▶ Define a subclass named K that extends the abstract class J and implements the interface I.
 - ▶ Both the iMethod1() and jMethod1() method prints the method name and the value of the parameter.

```
K value = new K();  
value.iMethod1(99);  
value.jMethod1(30);
```

```
iMethod1(), 99  
jMethod1(), 30
```



Exercise 2

```
interface Area {  
    void printArea(double a, double b);  
}
```

► Consider the following definition:

- Define a subclass named Rectangle
 - implements the Area interface
 - implements the printArea(double a, double b) method.
 - prints the area of a rectangle.
- Define a subclass named Circle
 - implements the Area interface
 - implements the printArea(double a, double b) method.
 - prints the area of a circle.

```
Area r1 = new Rectangle();  
r1.printArea(10, 5);  
Area r2 = new Circle();  
r2.printArea(2, 2);
```

```
The area of a Rectangle is 50.00  
The area of a Circle is 12.57
```



5. Extending Interfaces

- ▶ Interfaces can extend other interfaces, which gives rise to **sub-interfaces** and **super-interfaces**
 - ▶ Interface extends interface, but a class **cannot** extend an interface.
- ▶ Unlike classes, however, an interface can extend more than one interfaces at a time

```
public interface Displayable extends Drawable, Printable {  
    //additional constants and abstract methods  
    ...  
}
```
- ▶ Another good reason to extend interfaces is that **INTERFACES CANNOT GROW**.
 - ▶ Think about when you add methods to a class that's already being used by other classes. It doesn't effect them really, they can still use your class just as before.
 - ▶ But, if you add methods to an interface, you break the classes that use this interface, since they now must implement another method.
 - ▶ So, when something else is required in an interface, it is best to extend that interface, add the new method and then use the new sub-interface.



5. Extending Interfaces

Example 4

The new method

```
public interface Bounceable {  
    double GRAVITY = 9.8;  
    void hitWall(String wallName);  
    void AddMethod();  
}
```

- ▶ All classes that implement the old Bounceable interface will break because they don't implement all methods of the interface anymore.

```
public interface Bounceable {  
    void hitWall(String wallName);  
    ...  
}
```

```
public class Basketball implements Bounceable {  
    public void hitWall(String wallName) {  
        System.out.println("I hit " + wallName);  
    }  
    ...  
}
```

No AddMethod is defined in the Basketball. Compile-time error

- ▶ Solution: write a new sub-Interface

- ▶ Now users of your code can choose to continue to use the old interface or to upgrade to the new interface

```
public interface Bounceable {  
    double GRAVITY = 9.8;  
    void hitWall(String wallName);  
}
```

```
public interface BounceablePlus extends Bounceable {  
    void AddMethod();  
}
```

```
public class BasketballNew implements BounceablePlus {  
    public void hitWall(String wallName) {  
        ...  
    }  
    public void AddMethod() {  
        ...  
    }  
}
```




Exercise 3

- ▶ Consider the following definitions:

```
interface A {  
    void method1();  
    void method2();  
}  
interface B extends A {  
    void method3();  
}
```

- ▶ Define a subclass named MyClass which implements both interfaces.
 - ▶ method1() prints “implement method1”
 - ▶ method2() prints “implement method2”
 - ▶ method3() prints “implement method3”

```
MyClass c = new MyClass();  
c.method1();  
c.method2();  
c.method3();
```

```
implement method1  
implement method2  
implement method3
```

Exercise 4

- ▶ Given the following two interfaces:

```
interface Cowboy {  
    void draw(); // draw the gun - get ready to pump some iron  
    void drink(); // enjoy some beverage  
    void burp(); // communicate with other people in the bar (or where ever)  
}
```

```
interface Drawable {  
    void draw(int x, int y); // draw the drawable item on the screen  
}
```

- ▶ Define a class named person which implements the two interfaces
 - ▶ The burp() method prints "BURP !!"
 - ▶ The drink() method prints "I'm drinking..."
 - ▶ The draw() method prints "I'm drawing my gun"
 - ▶ The draw(x,y) method prints "I'm drawing a point at (x, y)"

Exercise 5

- ▶ Consider the following class definition:

```
class Person {  
    private String name;  
    private int age=1;  
    public Person(String n) {name = n;}  
    public Person(String n, int a) {name = n; age = a;}  
    public String toString() {return String.format("%s(%d)", name, age);}  
}
```

- ▶ Implement the **Comparable** interface to sort Person objects by name in alphabetical order

```
Person[] people = new Person[4];  
people[0] = new Person("Dick Smith", 21);  
people[1] = new Person("Anna Wong", 29);  
people[2] = new Person("Peter Kill", 32);  
people[3] = new Person("Emma Chen");  
Arrays.sort(people);  
System.out.println(Arrays.toString(people));
```

```
[Anna Wong(29), Dick Smith(21), Emma Chen(1), Peter Kill(32)]
```

Exercise 6

- ▶ Consider the following class definition:

```
class Rectangle {  
    private int width=1,height=1 ;  
    public Rectangle() {}  
    public Rectangle(int w, int h) {width = w; height = h;}  
    public int getArea() { return width * height; }  
    public String toString() {return String.format("Rectangle(%d)",getArea());}  
}
```

- ▶ Implement the **Comparable** interface to sort Rectangle objects by area

```
Rectangle[] data = new Rectangle[4];  
data[0] = new Rectangle(2, 9);  
data[1] = new Rectangle(12, 5);  
data[2] = new Rectangle(3, 4);  
data[3] = new Rectangle(4, 5);  
Arrays.sort(data);  
System.out.println(Arrays.toString(data));
```

```
[Rectangle(12), Rectangle(18), Rectangle(20), Rectangle(60)]
```



6. Typing Rules

- ▶ The typing rules for interfaces are similar to those for classes.
 - ▶ A reference variable of interface type T can refer to an instance of any class that implements interface T or a sub-interface of T.
 - ▶ Through a reference variable of interface type T, methods defined by T and its super interfaces can be called.



```
class c1 implements I1 {
    public void m1() {}
    public void m2() {}
}
```

```
interface I1 {
    public void m1();
    public void m2();
}
```

```
interface I2 extends I1 {
    public void m3();
}
```

```
class c2 implements I2 {
    public void m1() {}
    public void m2() {}
    public void m3() {}
}
```

```
C1 a = new C1();
C1 b = new C1();
I1 p = a;
p.m1(); ✓
p.m2(); ✓
```

a is a reference variable of type C1
p is a reference variable of type I1

```
C2 c = new C2();
I2 q = c;
q.m1(); ✓ can comp
q.m2() ✓
q.m3() ✓
```

q is a reference variable of type I2

$I_1 = C_1$
 $m_1 \checkmark \ m_2 \checkmark \ m_3 \times$





6. Typing Rules

The instanceof operator

- ▶ You can use the instanceof operator to test an object to see if it implements an interface, before you invoke a method in this interface.
- ▶ This might improve readability and correctness.
- ▶ This might be a hack.
 - ▶ Where possible, you should extend classes and interfaces to obtain polymorphic behaviour, rather than making a runtime check.

```
if( b instanceof Bounceable ) {  
    b.hitWall( "Wall A" );  
}
```



7. Final variables, methods, classes

- ▶ The final keyword can be applied to prevent some of the inheritance effects
 - ▶ Final field: constant
 - ▶ Final argument: cannot change the data within the called method
 - ▶ Final method: cannot override method in subclasses
 - ▶ Final class: cannot be subclassed (all of its methods are implicitly final as well)

```
class ChessAlgorithm {  
    . . .  
    final void nextMove(ChessPiece pieceMoved, BoardLocation newLocation) {  
        . . .  
    }  
}
```



Exercise 7

► Consider the following:

```
interface FinancialAidEligible { }  
abstract class Person {  
    int ID;  
    public int getID() { return ID; }  
}  
class Student extends Person {}  
class Undergraduate extends Student implements FinancialAidEligible {}
```

► Which of the following statements is/are LEGAL

```
I. FinancialAidEligible p1 = new Undergraduate();  
II. FinancialAidEligible p2 = new FinancialAidEligible();  
III. FinancialAidEligible p3 = new Student();  
IV. FinancialAidEligible[] people = new FinancialAidEligible[10];
```